

Digital Health Record Platform

Full-Stack Engineering, Mobile OTP Auth, UI/UX & Django Backend Implementation Report

Prepared For: Manan Soni & Project Team

Date: September 01, 2026

Tech Stack: Flutter, Python (Django REST), PostgreSQL, Redis

GitHub Repo: github.com/MananS0ni/Health

1. Executive Summary & Vision

Today we successfully designed, developed, and deployed the complete end-to-end architecture for the **Digital Health Record Platform**. The platform addresses medical record fragmentation by establishing a patient-owned digital health locker. It eliminates manual paper report scanning by enabling automated diagnostic report ingestion directly from hospital (HIS) and laboratory (LIS) systems.

2. Frontend UI/UX & Patient Experience (Flutter + Web)

We completely simplified and redesigned the patient-facing interface to be **clean, visual, and intuitive** for non-technical users. Heavy technical walls of text were replaced with visual metric cards, clear status chips, and plain-language summaries.

Module / Feature	Description & Patient Experience
Mobile OTP Authentication	Secure login via 6-digit SMS OTP code sent directly to patient's mobile number (+91 98765 43210).
Emergency Health Card	1-tap access card showing Blood Group (O+), Allergies, Emergency Contact, and scannable QR code.
Digital Health Locker	Central store for Blood Tests, Prescriptions, Scans & Hospital Notes with 1-click 'Explain in Simple Language'.
Unified Medical Timeline	Chronological history of medical visits across Dr. Lal PathLabs, Max Healthcare, Apollo, and Fortis.
1-Click Doctor Consent	Patient-controlled privacy manager with time-bound access limits and instant 'Stop Doctor Access' button.
Lab Auto-Sync Notice	Visual 3-step story showing how reports arrive automatically with zero manual upload effort.

3. Python Django REST Framework Backend Server

We built a modular, production-ready **Django REST Framework (DRF)** backend server under D:\Health\backend. It provides secure RESTful APIs, dual PostgreSQL/SQLite database support, SimpleJWT token authentication, and CORS cross-origin headers.

Django App	API Endpoint Route	Functionality
accounts	/api/auth/otp/verify/	Mobile OTP verification and JWT access/refresh token issuance.
patients	/api/patients/me/	Patient profile management, blood group, allergies & Emergency Card payload.
reports	/api/reports/	Fetches health locker documents & provides AI plain-language explanation.
consent	/api/consent/revoke/	Manages time-bound permissions and processes 1-click access revocation.
timeline	/api/timeline/	Streams chronological medical encounter history.
integrations	/api/integrations/lab/push/	Webhook endpoint allowing LIS lab systems to auto-push report payload.

Verified Live Backend API Response (<http://127.0.0.1:8000/api/integrations/status/>):

```
{"status": "HEALTHY", "service": "Django REST Framework Engine", "database": "Active"}
```

4. Code Repository & GitHub Deployment

The entire codebase was initialized with Git, configured for remote **MananS0ni**, and pushed to GitHub.

GitHub Repository URL: <https://github.com/MananS0ni/Health>

5. Quick Terminal Commands Reference

Run Flutter App: `cd D:\Health\health_platform && flutter run -d chrome`

Run Django Backend: `cd D:\Health\backend && .\env\Scripts\python.exe manage.py runserver 0.0.0.0:8000`

Push Code Updates: `cd D:\Health\health_platform && git push -u origin main`